Smart Parking Management System

Below is a complete Mini Project report structure based on your college PDF requirements. The PDF specifically lists Smart Parking Management as a suitable topic and says the system should manage parking slots, vehicle entry/exit, reservations, availability, and payments.

1. Title

SMART PARKING MANAGEMENT SYSTEM

Technology: MySQL + Python/Java/Web Interface

Database: MySQL

---

2. Introduction

The Smart Parking Management System is a database-driven application designed to manage parking facilities efficiently.

In a traditional parking system, information about available parking slots, vehicles, reservations, entry and exit times, and payments may be maintained manually. This can cause problems such as incorrect records, difficulty finding vacant slots, duplicate reservations, and delays in payment processing.

The proposed Smart Parking Management System stores all parking-related information in a centralized MySQL database. Users can check parking availability, reserve a parking slot, register vehicles, record entry and exit, and make payments.

The system also allows administrators to manage parking areas, parking slots, users, vehicles, reservations, and payments.

---

3. Problem Statement

Managing a large parking facility manually is difficult and time-consuming. The system needs to maintain accurate information about:

Parking areas

Parking slots

Registered users

Vehicles

Reservations

Vehicle entry and exit

Payments

Parking availability

Therefore, a computerized database system is required to manage parking operations efficiently.

---

4. Objectives

The main objectives are:

1. To maintain parking information digitally.

2. To identify available and occupied parking slots.

3. To allow users to reserve parking slots.

4. To maintain vehicle entry and exit records.

5. To calculate parking charges.

6. To maintain payment records.

7. To prevent duplicate parking reservations.

8. To generate parking reports.

9. To provide searching functionality.

10. To maintain data securely using MySQL.

---

5. Modules of the System

The system can contain the following modules:

5.1 User Management

Stores information about customers using the parking facility.

5.2 Vehicle Management

Stores vehicle details such as registration number, type and owner.

5.3 Parking Area Management

Stores information about different parking areas.

5.4 Parking Slot Management

Maintains individual parking slots and their availability.

5.5 Reservation Management

Allows users to reserve available parking slots.

5.6 Entry/Exit Management

Records vehicle entry and exit times.

5.7 Payment Management

Records parking fees and payment status.

5.8 Report Management

Generates reports about parking usage, reservations and payments.

---

6. Database Tables

The system uses 8 related tables, satisfying the minimum table requirement in your project PDF.

No.	Table	Purpose
1	Users	Stores user information
2	Vehicles	Stores vehicle information
3	Parking_Areas	Stores parking area details
4	Parking_Slots	Stores individual slots
5	Reservations	Stores slot reservations
6	Parking_Records	Stores vehicle entry/exit
7	Payments	Stores payment details
8	Parking_Rates	Stores parking charges

---

7. Database Design

Users

Field	Data Type	Key
user_id	INT	PK
name	VARCHAR(100)	
email	VARCHAR(100)	UNIQUE
phone	VARCHAR(15)	
password	VARCHAR(100)	

Vehicles

Field	Data Type	Key
vehicle_id	INT	PK
user_id	INT	FK
vehicle_number	VARCHAR(20)	UNIQUE
vehicle_type	VARCHAR(30)	
model	VARCHAR(50)	

Parking_Areas

Field	Data Type	Key
area_id	INT	PK
area_name	VARCHAR(100)	

location VARCHAR(150)

total_slots INT

Parking_Slots

Field	Data Type	Key
slot_id	INT	PK
area_id	INT	FK
slot_number	VARCHAR(20)	
slot_type	VARCHAR(30)	
status	VARCHAR(20)	

Reservations

Field	Data Type	Key
reservation_id	INT	PK
user_id	INT	FK
vehicle_id	INT	FK
slot_id	INT	FK
reservation_date	DATE	
start_time	DATETIME	
end_time	DATETIME	
status	VARCHAR(20)	

Parking_Records

Field	Data Type	Key
record_id	INT	PK
vehicle_id	INT	FK
slot_id	INT	FK
entry_time	DATETIME	
exit_time	DATETIME	
duration	DECIMAL(5,2)	

Payments

Field	Data Type	Key
payment_id	INT	PK
reservation_id	INT	FK
amount	DECIMAL(10,2)	
payment_date	DATETIME	
payment_method	VARCHAR(30)	
payment_status	VARCHAR(20)	

Parking_Rates

Field	Data Type	Key
rate_id	INT	PK
slot_type	VARCHAR(30)	
hourly_rate	DECIMAL(10,2)	
daily_rate	DECIMAL(10,2)	

---

8. Relationships

The main relationships are:

Users
 |
 | 1:M
 ↓
Vehicles
 |
 | 1:M
 ↓
Parking_Records
 |
 | M:1
 ↓
Parking_Slots
 |
 | M:1
 ↓
Parking_Areas

Users
 |
 | 1:M
 ↓

Reservations

↑ ↑

| |

Vehicles Parking_Slots

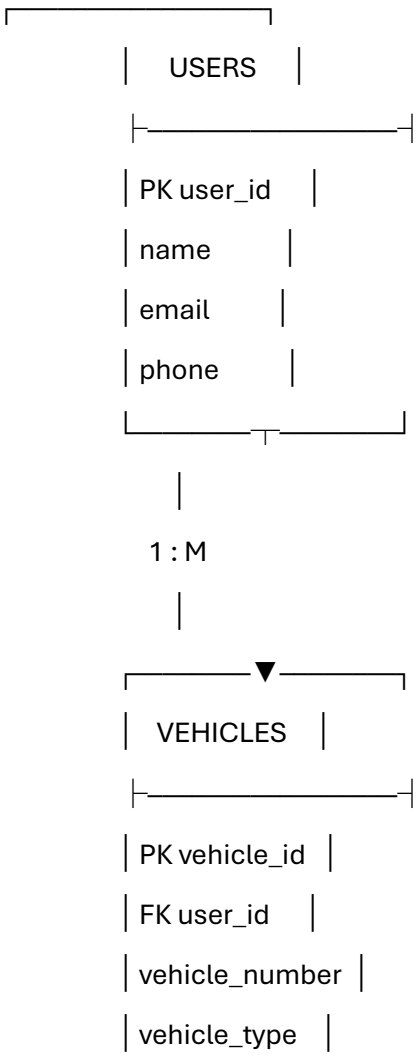
|

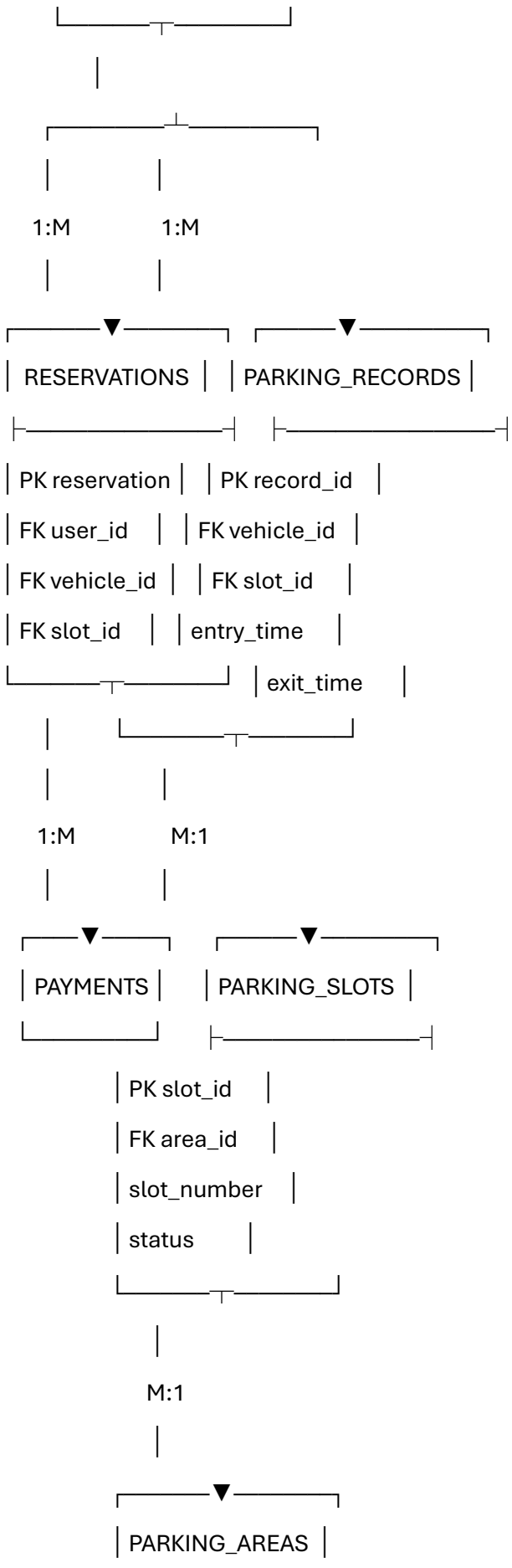
↓

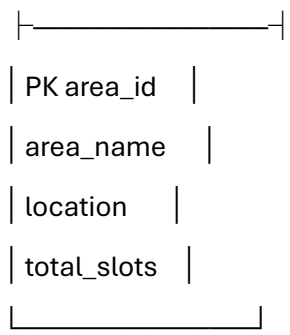
Payments

---

9. ER Diagram




1:M
1:M
RESERVATIONS
PK reservation
FK user_id
FK vehicle_id
FK slot_id
PARKING_RECORDS
PK record_id
FK vehicle_id
FK slot_id
entry_time
exit_time
1:M
M:1
PAYMENTS
PARKING_SLOTS
PK slot_id
FK area_id
slot_number
status
M:1
PARKING_AREAS



---

10. Relational Schema

```
USERS(
   user_id PK,
   name,
   email UNIQUE,
   phone,
   password
)

VEHICLES(
   vehicle_id PK,
   user_id FK,
   vehicle_number UNIQUE,
   vehicle_type,
   model
)

PARKING_AREAS(
   area_id PK,
   area_name,
```

location,

total_slots

)

PARKING_SLOTS(

slot_id PK,

area_id FK,

slot_number,

slot_type,

status

)

RESERVATIONS(

reservation_id PK,

user_id FK,

vehicle_id FK,

slot_id FK,

reservation_date,

start_time,

end_time,

status

)

PARKING_RECORDS(

record_id PK,

vehicle_id FK,

slot_id FK,

entry_time,

exit_time,

duration

)

PAYMENTS(

payment_id PK,

reservation_id FK,

amount,

payment_date,

payment_method,

payment_status

)

PARKING_RATES(

rate_id PK,

slot_type,

hourly_rate,

daily_rate

)

---

11. Primary Keys

Primary keys uniquely identify records.

Examples:

Users → user_id

Vehicles → vehicle_id

Parking_Areas → area_id

Parking_Slots → slot_id

Reservations → reservation_id

Parking_Records → record_id

Payments → payment_id

Parking_Rates → rate_id

---

12. Foreign Keys

Foreign keys create relationships between tables.

Examples:

Vehicles.user_id

↓

Users.user_id

Parking_Slots.area_id

↓

Parking_Areas.area_id

Reservations.user_id

↓

Users.user_id

Reservations.vehicle_id

↓

Vehicles.vehicle_id

Reservations.slot_id

↓

Parking_Slots.slot_id

Payments.reservation_id

↓

Reservations.reservation_id

---

13. Constraints

The database uses:

PRIMARY KEY

FOREIGN KEY

NOT NULL

UNIQUE

CHECK

DEFAULT

Example:

status VARCHAR(20) DEFAULT 'Available'

vehicle_number VARCHAR(20) UNIQUE

amount DECIMAL(10,2) CHECK(amount >= 0)

---

14. Normalization up to 3NF

First Normal Form – 1NF

All fields contain atomic values.

Example:

vehicle_number = "MH20AB1234"

Instead of storing multiple vehicle numbers in one field.

Second Normal Form – 2NF

The tables are in 1NF and every non-key attribute depends completely on the primary key.

For example, vehicle details depend on vehicle_id.

Third Normal Form – 3NF

The tables are in 2NF and non-key attributes do not depend on other non-key attributes.

For example, parking-area information is separated into Parking_Areas rather than repeatedly storing area information inside Parking_Slots.

---

15. MySQL Database Creation

```
CREATE DATABASE smart_parking;

USE smart_parking;
```

Users Table

```
CREATE TABLE Users (
  user_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  phone VARCHAR(15),
  password VARCHAR(100) NOT NULL
);
```

Vehicles Table

```
CREATE TABLE Vehicles (
  vehicle_id INT PRIMARY KEY AUTO_INCREMENT,
  user_id INT NOT NULL,
  vehicle_number VARCHAR(20) UNIQUE NOT NULL,
  vehicle_type VARCHAR(30),
  model VARCHAR(50),

  FOREIGN KEY (user_id)
  REFERENCES Users(user_id)
);
```

Parking Areas

```
CREATE TABLE Parking_Areas (
   area_id INT PRIMARY KEY AUTO_INCREMENT,
   area_name VARCHAR(100) NOT NULL,
   location VARCHAR(150),
   total_slots INT CHECK(total_slots >= 0)
);
```

Parking Slots

```
CREATE TABLE Parking_Slots (
   slot_id INT PRIMARY KEY AUTO_INCREMENT,
   area_id INT NOT NULL,
   slot_number VARCHAR(20) NOT NULL,
   slot_type VARCHAR(30),
   status VARCHAR(20) DEFAULT 'Available',

   FOREIGN KEY (area_id)
   REFERENCES Parking_Areas(area_id)
);
```

Reservations

```
CREATE TABLE Reservations (
   reservation_id INT PRIMARY KEY AUTO_INCREMENT,
   user_id INT NOT NULL,
   vehicle_id INT NOT NULL,
   slot_id INT NOT NULL,
   reservation_date DATE NOT NULL,
```

```
    start_time DATETIME NOT NULL,
    end_time DATETIME NOT NULL,
    status VARCHAR(20) DEFAULT 'Booked',

    FOREIGN KEY (user_id) REFERENCES Users(user_id),
    FOREIGN KEY (vehicle_id) REFERENCES Vehicles(vehicle_id),
    FOREIGN KEY (slot_id) REFERENCES Parking_Slots(slot_id)
);
```

Parking Records

```
CREATE TABLE Parking_Records (
    record_id INT PRIMARY KEY AUTO_INCREMENT,
    vehicle_id INT NOT NULL,
    slot_id INT NOT NULL,
    entry_time DATETIME NOT NULL,
    exit_time DATETIME,
    duration DECIMAL(5,2),

    FOREIGN KEY (vehicle_id) REFERENCES Vehicles(vehicle_id),
    FOREIGN KEY (slot_id) REFERENCES Parking_Slots(slot_id)
);
```

Payments

```
CREATE TABLE Payments (
    payment_id INT PRIMARY KEY AUTO_INCREMENT,
    reservation_id INT NOT NULL,
    amount DECIMAL(10,2) CHECK(amount >= 0),
    payment_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    payment_method VARCHAR(30),
```

```
    payment_status VARCHAR(20) DEFAULT 'Pending',

    FOREIGN KEY (reservation_id)
    REFERENCES Reservations(reservation_id)
);
```

Parking Rates

```
CREATE TABLE Parking_Rates (
    rate_id INT PRIMARY KEY AUTO_INCREMENT,
    slot_type VARCHAR(30) NOT NULL,
    hourly_rate DECIMAL(10,2) CHECK(hourly_rate >= 0),
    daily_rate DECIMAL(10,2) CHECK(daily_rate >= 0)
);
```

---

16. CRUD Operations

INSERT

```
INSERT INTO Users
(name, email, phone, password)
VALUES
('Rahul Patil', 'rahul@gmail.com', '9876543210', '12345');
```

SELECT

```
SELECT * FROM Users;
```

UPDATE

```
UPDATE Parking_Slots
SET status = 'Occupied'
WHERE slot_id = 1;
```

DELETE

```
DELETE FROM Users
WHERE user_id = 1;
```

---

17. JOIN Query

Display user, vehicle and reservation information:

```
SELECT
   u.name,
   v.vehicle_number,
   r.reservation_date,
   r.status
FROM Users u
JOIN Vehicles v
   ON u.user_id = v.user_id
JOIN Reservations r
   ON v.vehicle_id = r.vehicle_id;
```

---

18. Subquery

Find users who have made a reservation:

```
SELECT name
FROM Users
WHERE user_id IN (
  SELECT user_id
  FROM Reservations
);
```

---

19. View

Create a view for available parking slots:

```
CREATE VIEW Available_Slots AS
SELECT
  ps.slot_id,
  pa.area_name,
  ps.slot_number,
  ps.slot_type
FROM Parking_Slots ps
JOIN Parking_Areas pa
  ON ps.area_id = pa.area_id
WHERE ps.status = 'Available';
```

Use the view:

```
SELECT * FROM Available_Slots;
```

---

20. Trigger

The trigger can automatically mark a parking slot as occupied when a parking record is created.

```
DELIMITER //

CREATE TRIGGER after_parking_entry
AFTER INSERT ON Parking_Records
FOR EACH ROW
BEGIN
  UPDATE Parking_Slots
  SET status = 'Occupied'
  WHERE slot_id = NEW.slot_id;
END //

DELIMITER ;
```

---

21. Stored Procedure

A procedure can display available parking slots.

```
DELIMITER //
```

```
CREATE PROCEDURE GetAvailableSlots()
BEGIN
  SELECT
    slot_id,
    slot_number,
    slot_type,
    status
  FROM Parking_Slots
  WHERE status = 'Available';
END //

DELIMITER ;
```

Execute:

```
CALL GetAvailableSlots();
```

---

22. Search Functionality

Search for a vehicle:

```
SELECT *
FROM Vehicles
WHERE vehicle_number LIKE '%MH20%';
```

Search available slots:

```
SELECT *
FROM Parking_Slots
WHERE status = 'Available';
```

---

23. Parking Report

A parking report can display vehicle, slot and entry/exit information:

```
SELECT
   v.vehicle_number,
   ps.slot_number,
   pr.entry_time,
   pr.exit_time,
   pr.duration
FROM Parking_Records pr
JOIN Vehicles v
   ON pr.vehicle_id = v.vehicle_id
JOIN Parking_Slots ps
   ON pr.slot_id = ps.slot_id;
```

---

24. Sample Output

```
========================================
     SMART PARKING MANAGEMENT
========================================
```

Available Parking Slots

Slot ID	Slot No.	Type	Status
1	A-01	Car	Available
2	A-02	Car	Occupied
3	A-03	Bike	Available
4	A-04	Car	Available

==========

---

25. Advantages

1. Reduces manual parking management.

2. Provides quick information about available slots.

3. Reduces duplicate reservations.

4. Maintains accurate vehicle records.

5. Makes payment management easier.

6. Provides useful reports.

7. Centralizes all parking information.

8. Makes searching faster.

---

26. Limitations

1. Requires a database server.

2. Requires proper user authentication.

3. Internet/network connectivity may be required depending on the implementation.

4. Incorrect data entry can affect reports.

5. Hardware-based automatic vehicle detection is not included in the basic database implementation.

---

27. Future Scope

The system can be enhanced with:

QR-code parking tickets

Online payment

Mobile application

Automatic number-plate recognition

Real-time parking sensors

GPS-based parking location

SMS/email notifications

Digital parking receipts

Admin dashboard

Real-time parking availability

---

28. Conclusion

The Smart Parking Management System provides a structured database solution for managing parking facilities. The system maintains users, vehicles, parking areas, parking slots, reservations, parking records, payments and parking rates.

Using MySQL, the system can perform CRUD operations, joins, subqueries, views, triggers, stored procedures and reporting. The database design is normalized up to 3NF to reduce unnecessary data duplication and improve data consistency.

This project demonstrates how a database management system can be used to organize and manage a practical real-world parking application efficiently.

---

29. Technologies Used

Technology	Purpose
MySQL Database	
Python / Java / Web	Application interface
SQL	Database operations
HTML/CSS	Optional web interface
MySQL Workbench	Database design and testing

This structure covers the requirements listed in your assignment PDF, including 8 related tables, ER diagram, relational schema, keys, constraints, CRUD, joins/subqueries, 3NF, view, trigger, stored procedure, search/report functionality, and MySQL connectivity.